

Weave Engine

An Event-Driven, Decentralized Knowledge Runtime

Durable events. Reactive adapters.
Independent local knowledge.
Reconnection without implicit agreement.

VERSION 0.1 • 11 SEPTEMBER 2026

Prepared for Luis Palomo / Working title: Weave

ABSTRACT

The Weave Engine is a proposed local-first runtime for multidimensional temporal knowledge graphs. Its kernel commits first-class nodes, edges, assertions, and graph-valued metadata; a durable event bus then dispatches occurrences to registered, capability-scoped adapters. Adapters maintain derived views, cluster knowledge, synchronize peers, and request controlled external effects without bypassing the kernel. The architecture supports linked manifestations across spaces, native 3D and high-dimensional vector representations, lazy multiscale clustering, portable graph capsules, and offline mobile or personal-compute branches. This paper specifies commit-to-event atomicity, adapter lifecycle, replay and idempotency boundaries, selective replication, temporal reconciliation, visibility, and optional governance. It concludes with failure contracts, an end-to-end scenario, and an implementation and validation plan.

A proposed design, not an implementation or a benchmark report.

Contents

The bus connects reactions. The kernel protects knowledge semantics.

1. Executive architecture and scope	3
2. Kernel data model and graph-valued metadata	3
3. The event bus as the execution backbone	4
4. Adapter registration and execution contracts	7
5. Query execution and incremental materialization	8
6. Multidimensional identities and geometry	9
7. Automatic multiscale clustering and zoom	9
8. Distribution, capsules, attachment, and detachment	10
9. Offline branches, convergence, and temporal history	11
10. Permissions, visibility, and optional governance	12
11. Reliability, observability, and resource management	13
12. End-to-end example: edge evidence on an offline phone	14
13. Validation plan and implementation sequence	14
References	16

THREE DESIGN COMMITMENTS

DURABILITY A committed local transition and its event outbox share one atomic boundary.

EXTENSIBILITY Registered adapters consume authorized events and submit validated commands.

AUTONOMY Spaces, branches, and replicas remain distinct; synchronization is not acceptance.

READERS Runtime engineers, distributed-systems designers, security reviewers, and AI-platform builders.

COMPANION Paper 01 / Weave Language

1. Executive architecture and scope

The Weave Engine is a proposed local-first runtime for multidimensional, temporal knowledge graphs. Its defining execution mechanism is an event bus: the kernel records changes, publishes events, and dispatches them to registered adapters that can derive knowledge, maintain indexes, synchronize peers, update interfaces, or request external actions.

The engine treats nodes and edges as first-class objects with metadata. Metadata values may themselves be graphs, with the same identity, temporal, permission, query, and replication capabilities as other graphs. This requirement shapes the storage model, dependency tracking, event taxonomy, and offline working-set format; it is not implemented as a generic JSON field added later.

The proposed runtime spans mobile devices, personal workstations, and cooperating servers. No central service is required to create local knowledge or execute supported local queries. Larger peers can supply additional storage and compute under explicit delegation. A deployment may be centralized operationally without changing the logical model, but a central coordinator is not required for every graph operation.

The semantic kernel does not depend on language models or transformers. Rule evaluators, clustering algorithms, vector indexes, optional models, and external connectors are pluggable services. The companion Weave language compiles to the kernel's versioned operation and query contracts; other clients may use the same contracts directly.

Status and claims. This is architecture proposal v0.1. No engine implementation, throughput benchmark, clustering-quality result, or general-intelligence capability is claimed. “Must” denotes a proposed conformance requirement. Algorithm choices, storage backends, and deployment profiles remain implementation decisions unless stated otherwise.

1.1 Core boundaries

The kernel validates and commits knowledge. The bus transports occurrences and schedules reactions. Adapters propose further work; they do not acquire unrestricted storage access. Queries select explicit snapshots. Replication exchanges authorized history; governance determines acceptance. Derived indexes and clusters can be rebuilt without replacing original evidence.

This separation is essential. A message queue is not the graph store, an event is not necessarily an accepted fact about the world, a synchronized claim is not automatically trusted, and a cluster label is not a substitute for its underlying evidence.

2. Kernel data model and graph-valued metadata

The engine shares the language's distinction between entity identity, manifestation identity, edge identity, assertion identity, graph identity, and immutable revision identity. An entity can appear in several spaces. A manifestation holds space-local state and an authorized link to the shared identity. An edge connects typed manifestations, has its own ID, and can carry scalar or graph-valued metadata independently of its endpoints.

Structural edge identity fixes endpoints and predicate. Retargeting a relationship produces a new edge identity. Assertions record support, refutation, assumptions, provenance, valid time, and revision history. Multiple sources can make different assertions about one relationship without being collapsed into one mutable truth field.

A metadata attachment is an addressable record containing a host object reference, a field key, a typed value, and assertion context. Hosts include entities, manifestations, edges, and graph objects. Its value

may be a pinned graph reference or an explicitly declared live graph handle. Pinned references retain old meaning when the referenced graph advances.

ILLUSTRATIVE TEXT

```
MetadataAttachment {  
  id, host_id, key, schema_revision,  
  value: Scalar | ObjectRef | ArtifactRef | GraphRef | LiveGraphRef,  
  valid_interval, origin_assertion, access_policy,  
  revision, causal_parents  
}
```

An edge's evidence graph might contain observations, documents, calibration relationships, and counterarguments. Those observations and relationships can themselves carry metadata graphs. This recursive expressiveness is supported through references, not infinite inlining.

2.1 Storage representation

A reference implementation should use an append-only local commit journal, immutable object-revision blocks, branch/head tables, and rebuildable indexes. A commit references its causal parents, actor, schema and policy revisions, changed object blocks, and a manifest of required event records. Content addressing is useful for immutable blocks and revision exchange; stable logical IDs remain separate from hashes.

IPFS's Merkle-DAG documentation provides a relevant foundation for content-linked immutable structures. The engine need not depend on IPFS, and content addressing alone supplies neither authorization nor availability. [E1]

The knowledge graph may contain cycles even if the storage history is acyclic. Within a snapshot, object records use logical references resolved through the snapshot manifest, avoiding recursive hashes of mutually referring objects. A metadata graph can therefore refer back to its host without making snapshot serialization impossible. Byte-level canonical encoding is required for interoperable revision hashes; hash equality is not an identity-resolution algorithm.

2.2 Transactions and retention

A local transaction can create nodes, edges, metadata graphs, and their attachments atomically within one store domain. Cross-peer references do not create cross-peer atomicity. A schema may require particular references to be locally materialized before acceptance; otherwise their unresolved status becomes part of the boundary manifest.

Graph roots, pinned snapshots, active branches, subscribed checkpoints, and retained capabilities define storage roots. Garbage collection must trace graph-valued metadata and shared references. Detaching one capsule or deleting one attachment must not collect a metadata graph still retained elsewhere. Cyclic objects unreachable from all retention roots may be collected; naive reference counting alone is insufficient for that proposed behavior.

Tombstones and causal anchors are retained under an explicit policy. A peer returning after compaction may need a fresh snapshot and a controlled rebase rather than unrestricted replay of ancient operations. Retention limits must be visible to clients before promising historical replay.

3. The event bus as the execution backbone

Every peer runs a logical bus. On a phone this may be an embedded durable dispatcher over the local journal. A server may bridge the same contract to a broker. Peers exchange selected events and revisions through adapters, but there is no requirement for one globally shared event stream.

The bus separates durable domain events from ephemeral telemetry. A committed node change, metadata rebind, accepted proposal, or attached capsule has durable significance. A render-frame

update or transient progress measurement may use a best-effort channel. Losing telemetry must never imply losing a committed graph operation.

Durable messaging systems already distinguish persistence, replay, and acknowledgment from transient publish/subscribe. NATS documents this distinction between Core NATS and JetStream. The engine adopts an explicit delivery contract rather than inheriting unspecified behavior from whichever broker is chosen. [E2]

3.1 Commit-to-event atomicity

The kernel validates a command against types, capabilities, constraints, current local heads, and applicable policy. It then commits the state transition and its durable event outbox in one local atomic boundary. Only afterward does the dispatcher publish the events to consumers. A rollback produces no successful domain event.

If the process crashes after commit but before publication, restart resumes from the durable outbox. If it crashes after publication but before recording consumer acknowledgment, the event may be delivered again. The outbox pattern has an established implementation precedent in Debezium's event-router documentation. [E3]

The engine does not write graph state and publish to an unrelated external broker as two uncoordinated operations while claiming atomicity. The external broker is a delivery projection of committed local records. Broker failure delays delivery; it does not retroactively undo a durable local commit.

3.2 Event envelope

A CloudEvents-compatible envelope supplies the basic interchange fields. Engine-specific fields identify the commit, branch, causal ancestry, producer version, policy context, and graph references. CloudEvents defines envelope semantics, not durable delivery or a global ordering guarantee. [E4]

ILLUSTRATIVE JSON

```
{
  "specversion": "1.0",
  "id": "commit-c184/event-2",
  "source": "urn:weave:peer:p7",
  "type": "org.weave.meta.graph-rebound.v1",
  "subject": "opaque-edge-alias-17",
  "datacontenttype": "application/json",
  "data": {
    "commit": "c184",
    "branch": "field-work",
    "attachment": "a91",
    "before_graph": {"id": "proof", "revision": "r6"},
    "after_graph": {"id": "proof", "revision": "r7"},
    "caused_by": "command-551"
  }
}
```

The example is a payload shape, not a complete production wire schema. A production event also resolves to authenticated commit evidence, a schema version, and authorized snapshot context. Its IDs and subject are scoped aliases where public identifiers would reveal private topology. Capabilities or bearer secrets must not be embedded in ordinary event payloads.

Adapters may register versioned application-event schemas and publish only within authorized namespaces. They cannot impersonate kernel commit or governance-acceptance events. Custom events that assert persisted application transitions pass through a kernel command and commit boundary; best-effort notifications are labeled separately.

Event identity is stable for the occurrence, conventionally a source plus event ID. Local delivery attempts and broker offsets are not new occurrences. Importing an origin event preserves that identity.

A peer may additionally emit a distinct local `RevisionIntegrated` or `ProposalAccepted` event, linked to the origin rather than impersonating it.

3.3 Event taxonomy

Family	Representative events	Intended consumers
Knowledge	<code>NodeCreated</code> , <code>EdgeCreated</code> , <code>AssertionRecorded</code>	Views, validation diagnostics, search
Metadata	<code>MetadataSet</code> , <code>MetaGraphRebound</code> , <code>GraphRevisionCommitted</code>	Dependency trackers, evidence indexers
Identity/space	<code>ManifestationAdded</code> , <code>IdentityLinkAccepted</code> , <code>BridgeRevised</code>	Cross-space resolvers, geometry services
Derived views	<code>ViewAdvanced</code> , <code>ClusterSplit</code> , <code>ClusterMerged</code>	Navigation, caches, analysis agents
Mobility	<code>CapsuleAttached</code> , <code>CapsuleDetached</code> , <code>BranchIntegrated</code>	Replication, availability tracking
Policy	<code>ProposalSubmitted</code> , <code>ProposalAccepted</code> , <code>CapabilityRevoked</code>	Governance and access refresh
Execution	<code>AdapterFailed</code> , <code>EffectOutcomeUnknown</code> , <code>CheckpointExpired</code>	Operators and reconciliation services

A metadata graph changing internally is not the same occurrence as its host attachment being rebound. The first emits a graph-revision event. Pinned hosts remain unchanged. Live bindings may receive a derived invalidation event with explicit dependency information. The bus must not fabricate a host mutation for every graph that happens to reference an updated shared graph.

Query completion and sampled traversal events can be enabled for observability or optional learning experiments. They are not mandatory durable events for every read: that would impose unnecessary volume and could expose sensitive query behavior. Time-window expiration and scheduled triggers are explicit engine occurrences with recorded clock policy, not hidden nondeterminism inside pure lenses.

3.4 Delivery, ordering, and checkpoints

The baseline durable contract is at-least-once delivery within declared retention, authorization, and resource limits. A consumer has a durable checkpoint, bounded in-flight work, acknowledgment policy, retry policy, and lag visibility. Offline consumers resume from an available checkpoint or receive an explicit expiration error and rebuild path.

Ordering is local to a declared stream partition or commit sequence. A transaction's events carry a commit ID and ordinal. Cross-peer causality is represented by revision parents and prerequisite references; there is no implicit total order across disconnected peers. Consumers that require causally closed input must wait for dependencies or report them as unresolved.

A slow consumer cannot force unbounded storage growth. The engine applies quotas, backpressure, suspension, and retention policies. It must surface a lost-replay window rather than silently advance the checkpoint over undelivered domain events. Policy revocation may intentionally end delivery, with disclosure of that condition limited to what the principal is allowed to know.

4. Adapter registration and execution contracts

An adapter is a versioned executable module registered with a manifest. It may ingest external observations, transform graphs, maintain an index, perform clustering, transport revisions, render views, or request external actions. Some adapters are deterministic; others interact with models, networks, or devices.

The adapter manifest identifies the publisher, artifact digest, configuration revision, compatible engine and event schemas, subscribed scopes, capabilities, output targets, execution limits, state/checkpoint needs, and effect class. Registration is authorized. Discovery of a module in a graph does not execute it.

ILLUSTRATIVE TEXT

```
AdapterManifest {  
  id: "evidence-indexer", version: "0.1",  
  artifact_digest: "...", configuration_revision: "cfg3",  
  subscribes: ["org.weave.meta.graph-rebound.v1"],  
  scopes: ["operations/evidence"],  
  capabilities: [read_evidence, propose_semantic_index],  
  delivery: durable_at_least_once,  
  dedupe: [event_source, event_id, adapter_version, config_revision],  
  resources: {memory_mb: 256, concurrency: 2},  
  replay_mode: deterministic_projection  
}
```

4.1 Lifecycle and isolation

The lifecycle is install, validate, authorize, start, checkpoint, pause or drain, upgrade, and remove. An upgrade pins the new artifact and migrates adapter state explicitly. Old checkpoints are not reused against incompatible state or event schemas without a declared migration. A rollback restores a compatible artifact/checkpoint pair.

Untrusted adapters run outside the kernel's trust boundary, preferably in a sandbox or isolated process with explicit host calls. They receive no raw database handle, global event subscription, ambient filesystem access, or unrestricted network access. A deployment can choose a trusted native adapter for performance, but must document that expanded trust boundary.

Adapter outputs are commands or proposals. The kernel rechecks them under effective capabilities and policy at execution time. A post-commit adapter cannot veto the original transaction. Deterministic admission checks belong in a bounded validation interface before commit; network-dependent approvals use a proposal workflow rather than blocking the commit path indefinitely.

4.2 Idempotency and local atomic effects

A durable handler receives an event, resolves the permitted pinned inputs, and computes a command batch. It supplies a stable idempotency key covering the source occurrence, adapter version, configuration, output scope, and relevant revision. The kernel can atomically commit those commands and a processed-event receipt in the same local store.

If the handler crashes after that commit but before acknowledgment, redelivery finds the receipt and does not create another logical effect. Operations writing multiple independent stores require their own transaction or reconciliation design; an acknowledgment alone does not establish exactly-once state change.

An intentional algorithm upgrade uses a new version namespace and produces a new derived view lineage. It does not rewrite the provenance of the old output. Stale asynchronous results use expected-input revisions or compare-and-set checks; obsolete results can be retained for audit without becoming the current materialization.

4.3 External effects and uncertain outcomes

For an external side effect, the engine first records an effect intent with an idempotency key and authorization context. The adapter dispatches it, records the response, and reconciles retries against the remote system's capabilities. A crash after the remote action but before local confirmation creates an unknown outcome until reconciled.

An email system, device endpoint, or third-party API may not support idempotency or status lookup. The engine must then require a bounded retry, operator review, or application-specific compensation policy. It cannot truthfully promise exactly-once delivery to arbitrary external systems. Automatic graph reasoning does not by itself authorize physical control or other high-impact effects.

4.4 Replay, loops, and failure handling

Projection replay rebuilds indexes from pinned historical inputs and disables external effects by default. A nondeterministic adapter records actual output and its model/tool artifacts. Re-running a newer model is a new computation, not historical replay. Effect-enabled replay requires explicit authorization and a separate execution identity.

Loop control combines causal-chain IDs, origin tags, adapter namespaces, change detection, bounded execution budgets, and circuit breakers. Self-origin filtering alone is insufficient because two adapters can trigger each other. No-op writes should not emit a new domain change. Repeated adapter failures use exponential backoff, jitter, a dead-letter queue, and visible suspension; these are proposed operational controls rather than guarantees of eventual success.

5. Query execution and incremental materialization

The query service accepts a typed plan plus principal, snapshot requirements, coverage policy, and budget. It resolves graph heads and live metadata handles into a fixed input manifest, enforces visibility, selects indexes, executes joins and rules, and produces a graph with provenance and coverage. A broker notification never substitutes for reading the referenced committed snapshot.

Primary indexes cover logical IDs, edge endpoints and predicates, graph membership, valid intervals, metadata keys, and revision ancestry. Optional services add spatial and vector indexes. Graph-valued metadata has a reverse-dependency index from graph revisions or live handles to attachments, materialized views, and active subscriptions. Large evidence graphs remain lazy until a query actually needs them.

Materializations advance from input changes. A shared metadata update can have large fan-out, so invalidation is batched and prioritized. An index carries a processed-input watermark. A query requiring read-your-writes can wait for that watermark, fall back to primary records, or return an explicit freshness failure; it must not silently use a stale index as current truth.

Differential dataflow is a relevant research foundation for incremental and iterative computation. Whether it is the best implementation for this engine's temporal, provenance, and permission semantics requires evaluation. [E5] No general constant-time claim is made: a small source change can invalidate many conclusions, and joins can produce large outputs.

Every result includes selected snapshots, schema and rule versions, bridge mappings, provenance roots, assumptions, unresolved references, approximation status, and policy context. Cache keys include the principal's effective visibility context or a verified equivalent policy class. A result computed from restricted evidence is not served through a public cache key.

6. Multidimensional identities and geometry

A space contains manifestations, local relationships, and optional geometry. A replicated identity-link structure connects manifestations of the same entity. Traversal exposes authorized counterpart links as virtual adjacency, even if physical storage uses normalized membership rather than pairwise duplicates.

A capability-scoped resolver handles identity candidates, accepted mappings, superseding decisions, and splits. The design supports independently created identities becoming linked later, but a peer's assertion that two entities are the same is not automatically trusted by every graph. Historical queries pin their mapping revision.

Private manifestations may use separate identifiers with protected equivalence links. The existence of an identity group or its cardinality may itself be restricted. The resolver must not expose a global count that includes undiscoverable counterparts. W3C's DID privacy considerations motivate avoiding unnecessary public correlation. [E6]

6.1 Vector-aware spaces and edges

Physical coordinates include frame, dimensionality, units, and valid time. An edge may carry a vector-valued annotation for flow, orientation, displacement, or another schema-defined quantity. Semantic edge direction and geometric direction remain independent. A screen layout must not change dependency direction.

Embedding spaces identify encoder, preprocessing, dimensionality, metric, and revision. A bridge can connect a node's embedding manifestation to its operational or physical manifestation. Coordinate transforms and learned cross-space mappings are separate adapter products, with explicit domains, direction, parameters, and uncertainty. Shared entity identity does not imply a universal mathematical mapping between spaces.

A 3D representation may be native geometry or a projection of a higher-dimensional space. Projection output records its source coordinates, algorithm, and revision. Similarity services use the declared authoritative space rather than assuming that a visually close pair in a lossy projection is truly nearest.

Geometry, embeddings, and learned mappings inherit applicable restrictions. Text-embedding inversion research has demonstrated recovery of source information for studied systems, so an embedding is not treated as an anonymized release by default. [E7]

7. Automatic multiscale clustering and zoom

The clustering service reacts to relevant graph events and maintains derived organizations. Its input is a visible graph snapshot plus a task perspective, temporal window, selected feature spaces, algorithm configuration, and resource budget. Its output is a versioned cluster graph, not a rewrite of source evidence.

A cluster node carries a metadata graph describing membership, contributing relationships, source revisions, quality measures, and lineage. An aggregate edge carries a metadata graph of contributing lower-level edges. The same nodes may participate in multiple overlapping organizations. A particular zoom path selects one navigational expansion without claiming that the entire knowledge space is one tree.

7.1 Unbounded depth, finite work

The service supports no fixed semantic maximum depth. It materializes levels lazily, expanding or aggregating while evidence and budgets permit. If there is no useful additional abstraction or evidence,

it stops and records that boundary. A finite input is not made infinitely informative by repeatedly clustering it.

Within one abstraction hierarchy, membership relations must be acyclic and each nontrivial aggregation must make declared progress, such as reducing represented objects or changing a meaningful abstraction level. Ordinary cross-links can remain cyclic. This prevents a cluster containing itself from producing endless zoom recursion.

7.2 Candidate organization strategy

An initial strategy uses local affected-region updates, periodic reconciliation, stable cluster identifiers, and split/merge hysteresis. Alternatives include topology-only communities, geometric partitions, embedding neighborhoods, and explicitly curated domain hierarchies. Leiden is a relevant community-detection primitive with documented connectivity properties, not a complete solution to all these requirements. [E8]

The service records algorithm version, input order or seed where relevant, constraints, and quality diagnostics. Independently computing peers may produce different valid organizations. Synchronizing evidence does not require global agreement on one layout or clustering. A governed organization can be published as an accepted view where users need a shared navigation structure.

7.3 Correctness of zoom and privacy

Semantic zoom replaces aggregates with members, subclusters, or supporting evidence. Geometric zoom changes the viewport. Rendering and query APIs expose which action occurred. A coarse relation such as “cluster A depends on cluster B” must define whether it means at least one contributing edge, a threshold, or a universal condition.

Approximate summaries can guide search but cannot certify exact exclusion without a sound bound. An exact query must refine a cluster or use an index that proves no matches are hidden inside. The result reports incomplete expansion and approximation explicitly.

Clusters are computed from authorized inputs or explicitly released aggregates. Filtering private members after building a globally informed summary can still leak names, counts, coordinates, or topology. Public cluster versions therefore cannot be derived from secret evidence merely because the final member list is filtered.

8. Distribution, capsules, attachment, and detachment

A capsule is a portable, signed working-set manifest plus authorized content. It identifies included graph revisions, spaces, metadata graphs, schemas, rule/model artifacts, policy anchors, branch ancestry, and external references. Membership selection is separate from trust and acceptance.

Attaching a capsule adds an addressable graph source. Replication fetches permitted content. Forking creates an independent lineage. Integration combines changes under defined merge and policy rules. Detachment removes an active mount or synchronization relationship. None of these silently merges entity identities, accepts all claims, or recalls copies already disclosed.

8.1 Peer protocol

Peers negotiate protocol and schema versions, authenticate their relationship, establish scoped discovery, compare permitted revision summaries, request missing blocks, verify received content, and record local integration results. Content synchronization and event notification are related but separate: an event may arrive before its referenced block, and processing must wait or expose the missing dependency.

No global catalog is required. Peers can use direct pairing, local discovery, invitations, or an optional rendezvous service. Relay and NAT-traversal infrastructure may be necessary operationally, but it does not become an authority over graph identity or truth. A peer serving encrypted blocks need not be authorized to read their contents.

Selective replication requires verifiable partial disclosure. A signed commit root can authenticate selected blocks through inclusion proofs, while undisclosed data remains encrypted or outside the recipient's manifest. Required schema and causal anchors must still be available. Randomized encryption and scoped discovery reduce equality and topology leakage; the exact cryptographic format requires a separate reviewed protocol specification.

8.2 Metadata closure and explicit gaps

A capsule builder walks the selected nodes, edges, and required metadata attachments under permissions and budget. It uses visited IDs to handle cycles. Policies specify which metadata graphs are required, optional, or reference-only; requesting "all metadata" does not override access control or storage limits.

If a required proof graph cannot be included, the capsule builder either fails a strict export or marks the intended computation unavailable. It must not silently detach an edge from the evidence on which a rule depends. An optional unavailable graph stays an explicit boundary reference.

Reachability is not the same as availability. A content hash says which block is needed, not whether any current peer will serve it. Replication policies should define desired copy count, authorized storage peers, retention, and backup behavior, without treating those intentions as guaranteed durability.

9. Offline branches, convergence, and temporal history

A mobile peer is a local engine with a bounded working set, not merely a cloud cache. It can record observations, create metadata graphs, add manifestations, execute supported queries, and run authorized adapters offline. A workstation can later perform larger computations over the same operation contract. Local-first research provides the foundation for independent local use followed by collaboration. [E9]

An offline branch retains its event journal and causal parents. Valid time is supplied as part of an assertion. Local receipt time records what a particular replica observed. Accepted views may record acceptance time. An author's wall-clock timestamp is not trusted evidence that an operation preceded a revocation or competing update.

9.1 Merge by semantic category

Data category	Proposed reconciliation behavior
Immutable assertions and evidence	Union by stable origin identity; preserve independent sources
Concurrent scalar metadata	Multi-value conflict unless a schema declares another safe rule
Concurrent graph metadata bindings	Preserve competing graph references; do not blindly merge contents
Identity-link proposals	Retain candidates; apply the selected acceptance policy
Derived indexes and layouts	Rebuild or version independently from authoritative input history
Exclusive governed decisions	Coordinate or keep proposals unaccepted until authority is available

CRDTs can provide convergence for selected structures, but convergence does not decide which real-world account is correct. Automerge's documented conflict behavior illustrates the distinction between retaining concurrent values and presenting one deterministic property value. [E10] The engine should expose semantic conflicts rather than disguise them as truth chosen by arrival time.

CRDT claims must identify the actual operation algebra, causal assumptions, tombstone rules, and supported invariants. Graph referential integrity and authorization do not become solved simply by using a CRDT library for maps. A concurrently deleted endpoint and newly added edge require an explicit dangling-edge or acceptance rule.

9.2 Reconnection and admission

Reconnection first exchanges and verifies authorized history. It then checks schema compatibility, causal dependencies, capability and policy context, and domain constraints. Finally, it admits changes into the target accepted view or retains them as proposals/conflicts. Remote rejection does not erase the author's local branch.

Some invariants require coordination. Research on invariant confluence formalizes when coordination-free execution can preserve application invariants and when it cannot. [E11] An exclusive ownership transfer or one-winner decision may therefore require a scoped authority or consensus process even while ordinary observation capture remains available offline.

Detachment is not deletion, and deleting a local copy is not a guaranteed network-wide erasure. Revocation can stop new authorized access and future admission, but cannot guarantee that a disconnected recipient forgets plaintext already received. The engine must make this boundary explicit in policy and user-visible state.

10. Permissions, visibility, and optional governance

The security model distinguishes discover, read, traverse, propose, publish, subscribe, execute, replicate, and delegate. Capabilities are resource- and operation-scoped, bounded by policy, and checked at use time. UCAN provides a relevant public-key capability-delegation design, but this engine does not assume that delegation alone solves revocation, confinement, or offline admission. [E12]

A subscription is itself a read capability. Event type, subject, timestamp, count, and topology can reveal information even without payload content. The dispatcher therefore filters before delivery, uses scoped envelopes where needed, and requires separate authorization when an adapter resolves a graph reference. Dead-letter queues, audit logs, and metrics inherit appropriate restrictions.

10.1 Graph-valued policy and information flow

Policy and governance documents may be represented as graphs, but active policy roots are installed through a bounded trusted mechanism. The kernel evaluates a pinned, authorized policy revision. Arbitrary metadata named `policy`, or code found inside a document, cannot grant authority or execute itself. Bootstrap trust must not require trusting an unverified graph that authorizes its own installation.

Reading through a private host's metadata attachment cannot reveal the host-to-graph relationship without permission. A graph independently published elsewhere can remain public through that other access path. Derived outputs track the actual evidence and policy context that influenced them, rather than globally relabeling every shared object.

Authorized declassification is an explicit operation producing a released result and audit record. An adapter with access to private input does not automatically obtain permission to publish summaries, embeddings, or transformed coordinates. Sandboxing and explicit egress policies are required, although a complete side-channel-resistant information-flow system remains a research and assurance task.

10.2 Optional governance

A personal graph may publish under owner authority. A team graph may require reviewer approval. A federation may maintain multiple accepted views over common evidence. Governance adapters can collect reviews and propose acceptance, but the kernel validates the applicable signed policy and decision proof before changing an accepted head.

Policy changes are versioned and authorized under the preceding policy. Threshold signatures or approvals require explicit membership and decision rules; they do not automatically prevent conflicting quorums or Sybil participation. Exclusive decisions require a scoped ordering or consensus mechanism. The whole graph network does not require a blockchain or universal consensus.

10.3 Threat model and limits

The baseline assumes potentially malicious peers, malformed capsules, duplicated messages, compromised adapters, stale capabilities, poisoned evidence, and resource-exhaustion attempts. Controls include signature and hash checks, schema validation, capability attenuation, quarantined imports, resource budgets, isolated execution, and explicit source acceptance.

A signature proves attribution to a key, not correctness of a claim. Encryption does not protect plaintext on a compromised authorized endpoint. Hardware trust, key recovery, cryptographic erasure, and metadata-traffic resistance need deployment-specific designs and review. This paper is not a completed security proof or certification claim.

11. Reliability, observability, and resource management

The engine exposes commit durability, replication lag, consumer lag, index freshness, unresolved dependencies, stale derived views, rejected proposals, and unknown external-effect outcomes. Operators and agents need these states to distinguish “no result” from “not evaluated” and “committed locally” from “accepted remotely.”

Backpressure applies independently to ingestion, durable event delivery, adapter execution, metadata expansion, and peer synchronization. Per-principal budgets prevent one graph with recursive metadata or a high-fan-out update from monopolizing a device. Mobile scheduling can defer expensive work based on power and memory policy while keeping local capture and basic queries available.

Failure	Required behavior
Crash after graph commit, before bus publication	Resume durable outbox publication
Redelivery after handler effect commit	Reuse idempotency receipt; no duplicate logical local effect
Referenced metadata graph unavailable	Defer or report incomplete coverage, never fabricate emptiness
Slow index adapter	Expose watermark lag; wait, fall back, or fail freshness requirement
Adapter feedback loop	Bound execution and suspend the chain with diagnostics
External action response lost	Record unknown outcome and reconcile before unsafe retry
Peer returns beyond retention horizon	Require explicit snapshot/rebase, not silent history loss
Capability revoked during disconnect	Apply declared admission policy when reconnecting

Audit records link commands, commits, events, adapter invocations, output proposals, and external-effect intents. Sampling may reduce nonessential query telemetry, but committed mutation provenance and security-critical audit retention follow explicit policy. Logs are not a permission-free copy of the graph.

Storage and query implementations should be benchmarked separately from transport choices. A reference prototype can use an embedded transactional store on personal devices and a broker bridge on servers. Choosing a particular database, broker, sandbox, or vector index is deferred until measured against the common conformance contract.

12. End-to-end example: edge evidence on an offline phone

A user opens an installation graph on a phone and selects a working capsule containing operational and physical manifestations of a device, a gateway, their connection edge, and the edge's evidence graph. Optional private annotations remain in a separate space. Required schemas and causal anchors are included before detachment.

Offline, a new measurement contradicts the expected connection quality. One local transaction creates a measurement node, adds its source relationship in a new evidence-graph revision, and rebinds the connection edge's evidence metadata. It records the old and new graph references and commits the corresponding durable events.

The bus delivers MetaGraphRebound to an authorized diagnostic adapter. The adapter reads the pinned evidence graph and proposes a warning assertion. The kernel commits that proposal with the adapter's idempotency receipt. A cluster adapter receives the resulting change and creates a candidate failure-pattern cluster whose metadata records the contributing nodes and edges. None of this requires a remote service.

On reconnection, the phone's synchronization adapter exchanges authorized revisions with a workstation. The workstation verifies the history and emits a distinct local integration event. Its larger clustering service may produce a different organization over more permitted evidence, retaining the phone's proposal as a separate derived revision.

The team graph receives a proposal referencing the new evidence. A governance adapter gathers required review, and the kernel validates the decision before advancing the team's accepted view. The edge's prior evidence revision remains available for historical queries. Private annotations and inaccessible counterpart links are not exported merely because the operational capsule was attached.

An external maintenance notification is sent only through an authorized effect adapter. If the response is lost, its ledger records an unknown outcome rather than creating repeated notifications through unrestricted bus replay. This workflow demonstrates the relationship between graph-valued metadata, local events, reactive adapters, cross-space identity, offline branches, and optional acceptance.

13. Validation plan and implementation sequence

The initial kernel should implement identity, manifestations, typed edges, assertion history, pinned graph-valued metadata, transactions, and a durable outbox. The next stage adds a query API, deterministic adapter SDK, idempotent command receipts, and local replay. This creates a testable vertical slice before introducing complex federation or automatic organization.

Subsequent stages add capsules and two-peer offline integration; capability-scoped event delivery and governance; then incremental clustering, geometric/vector services, and workload-specific adapters. The language compiler should target the same operation protocol throughout. Avoid a prototype that stores metadata graphs in a special side database with incompatible permissions or history.

13.1 Required experiments

Correctness experiments should inject crashes at every commit/publication/acknowledgment boundary, reorder and duplicate peer messages, generate cyclic metadata graphs, and create concurrent graph-reference edits. Security tests should attempt subscription leakage, malicious identity links, unauthorized graph expansion, policy self-installation, and public derivatives influenced by private inputs.

Convergence tests compare authorized assertion sets and accepted-view rules after eventual delivery, not merely serialized bytes of cached layouts. Clustering tests measure membership stability, split/merge churn, quality for selected tasks, incremental-versus-full recomputation differences, and exact-search recall after zoom-based routing. Approximate methods must declare their error criterion.

Performance experiments should report hardware, data shape, edge density, metadata depth and sharing, adapter workload, disconnected duration, and visibility partitions. Measure median and tail latency, peak memory, bytes transferred, event amplification, recovery duration, and mobile energy cost. Suggested test scales are 10,000, 100,000, and 1,000,000 graph objects where hardware permits; these are workload targets, not demonstrated capacity.

13.2 Remaining research and engineering decisions

Open work includes a canonical partial-replication format, a precise graph CRDT/invariant model, efficient shared-metadata invalidation, provenance compression, identity split propagation, privacy-preserving multi-user clustering, and reproducibility of approximate geometric services. Formal models are needed for transaction/event atomicity, capability transitions, and replay safety. Adversarial testing and independent security review remain necessary before production trust claims.

A future reward-based traversal adapter could store versioned path preferences separately from evidence, provided repeated transport is not treated as repeated success and permission boundaries remain enforced. That is an optional experimental extension, not part of the core engine contract or a validated learning architecture.

Conclusion. The engine's organizing principle is a durable event-driven kernel around first-class knowledge objects. Nodes, edges, and their metadata graphs share one temporal and security model. Registered adapters extend behavior without bypassing commit semantics. Spaces provide linked manifestations; capsules and branches provide independent local operation; explicit integration and governance permit reconnection without confusing synchronization with agreement.

References

- [E1] IPFS documentation. *Merkle Directed Acyclic Graphs*. [Source](#)
- [E2] NATS documentation. *JetStream*. [Source](#)
- [E3] Debezium documentation. *Outbox Event Router*. [Source](#)
- [E4] CloudEvents project. *CloudEvents specification*, version 1.0.2. [Source](#)
- [E5] McSherry, F.; Murray, D. G.; Isaacs, R.; Isard, M. *Differential dataflow*. CIDR, 2013. [Source](#)
- [E6] W3C. *Decentralized Identifiers (DIDs) v1.0*, privacy considerations. [Source](#)
- [E7] Morris, J.; Kuleshov, V.; Shmatikov, V.; Rush, A. *Text Embeddings Reveal (Almost) As Much As Text*. EMNLP, 2023. [Source](#)
- [E8] Traag, V. A.; Waltman, L.; van Eck, N. J. *From Louvain to Leiden: guaranteeing well-connected communities*. 2019; preprint arXiv:1810.08473. [Source](#)
- [E9] Kleppmann, M.; Wiggins, A.; van Hardenberg, P.; McGranaghan, M. *Local-first software: You own your data, in spite of the cloud*. Onward!, 2019. [Source](#)
- [E10] Automerge documentation. *Conflicts*. [Source](#)
- [E11] Bailis, P.; Fekete, A.; Franklin, M. J.; Ghodsi, A.; Hellerstein, J. M.; Stoica, I. *Coordination Avoidance in Database Systems*. 2014. [Source](#)
- [E12] UCAN working group. *User Controlled Authorization Network specification*. [Source](#)

All online references were consulted on 11 September 2026. References identify technical foundations and limitations, not validation of the proposed engine. The companion document is *Weave: A Graph-Native Programming Language*, architecture proposal v0.1.